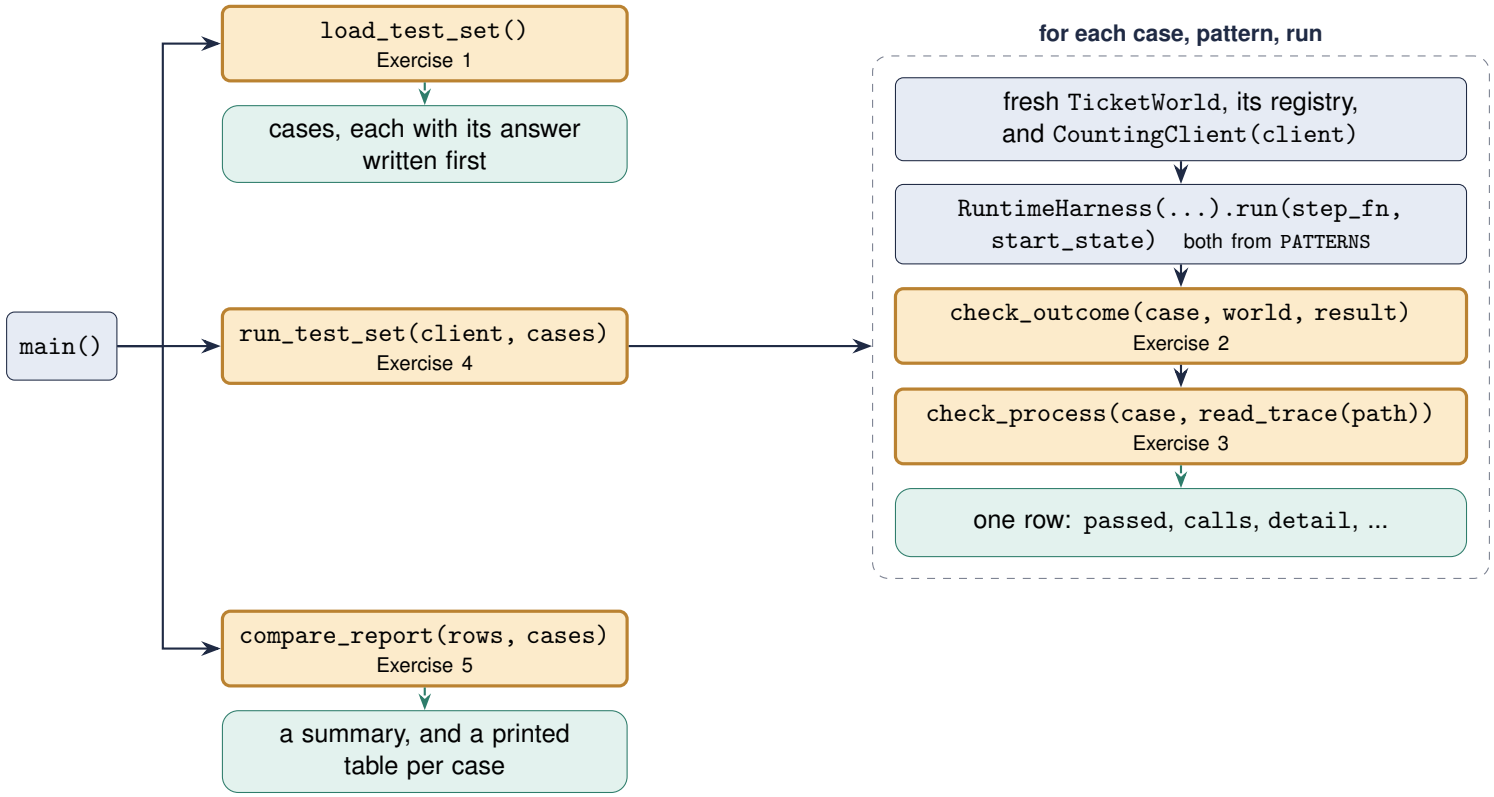
Lab 9a call flow, 1 of 5: who calls whom

`python3 scaffold.py` loads the tests, runs every case on every version, checks each run, and compares. Gold boxes are your exercises.



The agent is untouched. *`PATTERNS` holds the two versions under test: the lecture's `react_step` and your own Lab 8a `plan_execute_step`. The harness around them is Week 7's.*

Two checks, two sources. *The outcome check looks at the* world *after the run. The process check reads the* trace *file the harness wrote.*

Fresh per run. *A new world and a new `CountingClient` each time, so no run sees another's escalations, and `calls` counts only that run.*

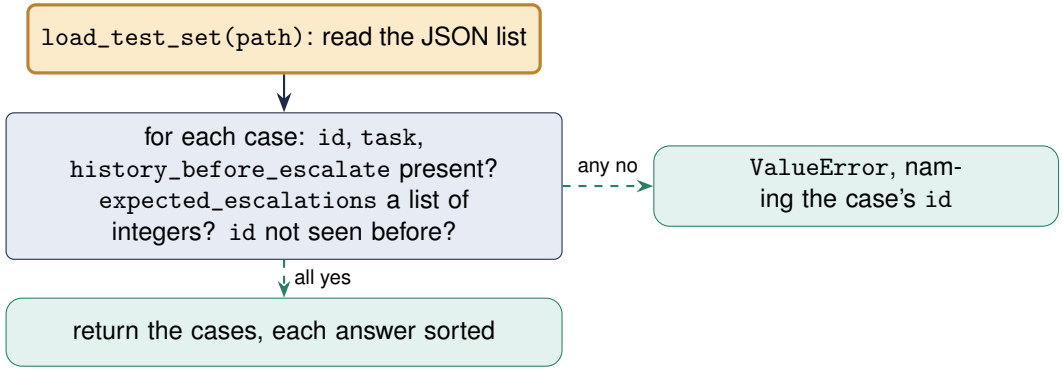
given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 9a call flow, 2 of 5: a test case, and `load_test_set`

Exercise 1. The answer is written before any agent runs. Your loader refuses a test set that is not ready.

```
{"id": "escalate-never-escalated",
 "task": "Find all unresolved tickets and escalate ...",
 "expected_escalations": [23, 42],
 "history_before_escalate": true}
```

***id**: a unique name.*
***task**: exactly what the agent is told.*
***expected_escalations**: the answer, worked out from `tickets.json` by a person.*
***history_before_escalate**: whether the process check applies.*



***Two cases ship with `null` answers.** Work them out from the data and write them into `test_set.json`. Until you do, your own loader refuses to run the tests. That is the point: no answer, no test.*

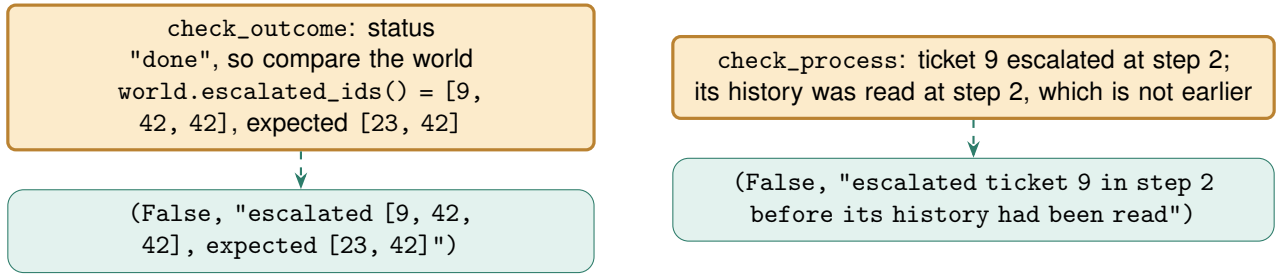
***Sorted on load**, so the outcome check can compare two sorted lists directly, and a repeated ticket shows up as a difference.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 9a call flow, 3 of 5: the two checks

Exercises 2 and 3. A real ReAct run of `escalate-never-escalated`, 4 October, and what each check says about it.

Step	The trace (`output` of each harness step)	Histories read so far
1	`search_tickets("unresolved")` → `[9, 23, 42]`	none
2	`get_ticket_history` 9, 23, 42 *and* `escalate_ticket` 9, 42	9, 23, 42 at step 2
3	`escalate_ticket` 42 again	
4	final answer (a string, not a list: no tool calls)	



The outcome check says* that *it is wrong: *the wrong ticket, and 42 twice. It never reads the trace.*

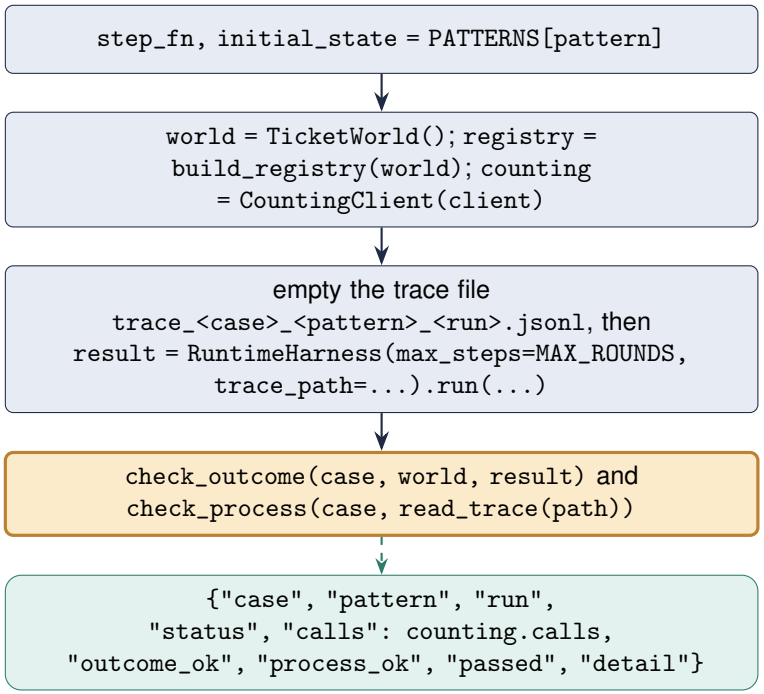
***The process check says* why**: *the escalation was decided in the same turn the histories were requested, before they came back.*

Only list outputs hold tool calls. *Step 4's output is the final answer, a string. Entries without a `"tool"` key, like a planner's `not_run` note, are skipped.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 9a call flow, 4 of 5: one run inside `run_test_set`

Exercise 4. This happens once for every case, pattern, and run, in that order.



The harness calls the step function with only `state`. The `counting` client, the registry, and `GRANTED_FULL` are tied in when you build the function you hand to `run`, as in Labs 7 and 8.

`passed` needs both checks. `detail` is `"ok"`, or the failing checks' details, each prefixed `"outcome: "` or `"process: "`, joined by `"; "`.

given code | you write this | outside your code | data passed along | calls | returns

Lab 9a call flow, 5 of 5: from rows to a comparison

Exercise 5. The real report from a full run, 4 October: every case, both versions, one run each.

Case	react		plan-execute	
escalate-never-escalated	0/1	4.0 calls	1/1	4.0 calls
known-path	1/1	2.0 calls	1/1	3.0 calls
older-than-six-days	0/1	3.0 calls	1/1	6.0 calls
resolved-ticket	1/1	2.0 calls	1/1	2.0 calls
already-escalated	1/1	2.0 calls	1/1	2.0 calls
missing-ticket	1/1	3.0 calls	1/1	2.0 calls
younger-than-six-days	0/1	3.0 calls	1/1	6.0 calls
only-if-unresolved	1/1	3.0 calls	1/1	3.0 calls
Total	**5/8**		**8/8**	

***The summary is nested**: `summary[pattern][case_id]` holds `passed`, `runs`, and `mean_calls`. The printed table is one view of it.*

***Prediction 1, checked.** ReAct failed exactly the cases where an escalation depends on a history it had not read yet. Your planner passed all eight.*

***And it cost more** where the path was known: 3 calls against 2. The report puts cost next to correctness, as the lecture said it should.*

given code | you write this | outside your code | data passed along | → calls | - - - → returns